

Uni-FACEF CENTRO UNIVERSITÁRIO MUNICIPAL DE FRANCA

**CAUÃ MENDONÇA MARTELLO
DIEGO HENRIQUE CUNHA DE MELO
GUILHERME SOUZA DE ANDREA
LUIS FELIPE FERRACINE
RODOLFO ZUCULU PATURALSKI**

**CONSTRUÇÃO DE SOFTWARE ORIENTADO A COMPONENTES
PARA UMA COMUNIDADE
UCE – PARADIGMAS DE PROGRAMAÇÃO II**

**FRANCA
2025**

**CAUÃ MENDONÇA MARTELLO
DIEGO HENRIQUE CUNHA DE MELO
GUILHERME SOUZA DE ANDREA
LUIS FELIPE FERRACINE
RODOLFO ZUCULU PATURALSKI**

**CONSTRUÇÃO DE SOFTWARE ORIENTADO A COMPONENTES
PARA UMA COMUNIDADE
UCE – PARADIGMAS DE PROGRAMAÇÃO II**

Relatório Final de Unidade Curricular de Extensão – UCE de Paradigmas de Programação II, apresentado ao Departamento de Computação do *Uni-FACEF - Centro Universitário Municipal de Franca*, para atender às disposições da Resolução nº 7 de 18 de dezembro de 2018, que estabelece as Diretrizes para a Extensão na Educação Superior.

Prof. Responsável: Prof. Dr. Daniel Facciolo Pires

**FRANCA
2025**

SUMÁRIO

1. INTRODUÇÃO	4
2. PROJETO DE UNIDADE CURRICULAR DE EXTENSÃO - UCE	4
2.1. OBJETIVO GERAL	4
2.1.1. OBJETIVOS ESPECÍFICOS	4
2.2. METODOLOGIA	4
3. ESTUDOS TEÓRICOS	5
4. IMPLEMENTAÇÃO DA SOLUÇÃO DO BACKEND	5
4.1. TECNOLOGIAS UTILIZADAS	Erro! Indicador não definido.
4.2. ARQUITETURA E ORGANIZAÇÃO DAS CAMADAS	Erro! Indicador não definido.
4.3. IMPLEMENTAÇÃO DOS MÓDULO	Erro! Indicador não definido.
4.3.1. Módulo: Empresa, Entidade Empresa	Erro! Indicador não definido.
5. IMPLEMENTAÇÃO DA SOLUÇÃO DO FRONTEND	13
7. CONSIDERAÇÕES FINAIS	20
8. CONTRIBUIÇÕES DA UCE PARA A FORMAÇÃO DISCENTE	20

1. INTRODUÇÃO

A idealização do projeto da UCE, surgiu de uma necessidade identificada na sociedade, onde foram ouvidas as dores de uma parcela da população que não tem reconhecimento e inclusão necessária. Assim, foi montado o escopo do projeto, onde podemos acolher pessoas com deficiência e dando a possibilidade de serem inseridas no mercado de trabalho.

Logo, o projeto de um sistema de vagas está em fase final de desenvolvimento, buscando a melhor solução sistêmica para inclui-los no âmbito profissional.

Ao criar aplicações na forma de software que apoiem as atividades de entidades sociais, ou pequenas e médias empresas, esta unidade de extensão explora a ODS 1 (erradicação da pobreza), a ODS 9 (indústria, inovação e infraestrutura), a ODS 10 (redução das desigualdades), a ODS 8 (trabalho decente e crescimento econômico) e a ODS 11 (cidades e comunidades sustentáveis). A depender do ramo de atuação das entidades e empresas, outras ODSs podem ser exploradas.

2. PROJETO DE UNIDADE CURRICULAR DE EXTENSÃO - UCE

2.1. OBJETIVO GERAL

O projeto tem como objetivo capacitar os discentes a desenvolver uma aplicação web completa, utilizando paradigmas de programação orientada a objetos, funcional e orientada a componentes.

2.1.1. OBJETIVOS ESPECÍFICOS

Os objetivos específicos são: desenvolver uma solução tecnológica com impacto social relevante, aplicar conceitos avançados de desenvolvimento web full-stack e trabalhar ODSs relacionadas à inclusão e desenvolvimento sustentável.

2.2. METODOLOGIA

Aulas expositivas em sala de aula. Aulas em laboratório de informática para criação de softwares. Interação e socialização, por meio de entrevistas, com os

responsáveis pela(s) entidade(s) social(is) ou empresa(s) de pequeno e médio porte(s), e ainda avaliação pelos coordenadores do produto apresentado.

3. ESTUDOS TEÓRICOS

1) RocketSeat. DevLinks Disponível na web em https://www.rocketseat.com.br/devlinks . Último acesso: março de 2023

Concepções/estudos empregados: Quais os recursos foram estudados e como estes auxiliam na criação do site.

2) W3Schools. JavaScript Tutorial. Disponível na web em https://www.w3schools.com/js/default.asp . Último acesso: 03/2023
--

Concepções/estudos empregados: Quais os recursos foram estudados e como estes auxiliam na criação do site.

3) Guanabara, G. Curso de HTML5 e CSS3. Disponível na web em https://www.youtube.com/playlist?list=PLHz_AreHm4dlAnJ_jJtV29RFxnPHDu_k9o . Último acesso: março de 2023

Concepções/estudos empregados: Quais os recursos foram estudados e como estes auxiliam na criação do site.

4) CARVALHO, Marly Monteiro De. Fundamentos em Gestão de Projetos: Construindo competências para gerenciar projetos, 2015

Concepções/estudos empregados: Gestão de projetos e suas competências
--

5) MAGEE, David. O SEGREDO DA TOYOTA: lições de liderança da maior fabricante de automóveis do mundo. Rio de Janeiro: Elsevier, 2008.

Concepções/estudos empregados: Visão empreendedora e analítica

4. IMPLEMENTAÇÃO DA SOLUÇÃO DO BACKEND

4.1. Tecnologias Utilizadas

O back-end foi construído utilizando Node.js como ambiente de execução, TypeScript para garantir tipagem estática e segurança no desenvolvimento, e Express.js como framework responsável pela definição das rotas e organização das requisições HTTP.

O TypeScript proporcionou maior confiabilidade ao código, evitando erros de tipagem, melhorando a legibilidade e facilitando a manutenção. Como o navegador não interpreta TypeScript diretamente, todo o código passou por um processo de transpilação para JavaScript.

Para comunicação com o banco de dados PostgreSQL, foi utilizada a biblioteca node-postgres (pg). Todas as operações — consultas, inserções, atualizações e exclusões — foram realizadas através de queries SQL escritas manualmente, garantindo maior controle sobre desempenho, otimização das operações e aderência às regras de negócio estabelecidas pelo sistema.

4.2. Arquitetura e Organização das Camadas

A aplicação foi estruturada seguindo uma divisão clara de responsabilidades, baseada nos princípios do modelo arquitetural MVC e inspirada em boas práticas de Clean Architecture:

1. Camada de Rotas (Routes)

Responsável por mapear os endpoints HTTP da aplicação. Nessa camada, apenas direcionamos a requisição ao controlador correspondente.

2. Camada de Controladores (Controllers)

Os controladores recebem as requisições vindas das rotas, realizam validações iniciais e acionam os serviços adequados. Entre os controladores implementados estão:

EmpresaController

VagaController

CandidatoController

MatchController

Essa camada intermedeia a comunicação entre rotas e regras de negócio.

3. Camada de Serviços (Model)

Aqui estão concentradas as regras de negócio. Os serviços realizam validações, tratam dados, chamam a camada de repositórios e executam operações complexas, como:

- criação de vaga;
- registro de candidato;
- cálculo de compatibilidade (match).

4. Camada de Repositórios (SQL)

Camada responsável pela comunicação direta com o banco de dados. Utiliza *node-postgres (pg)* para executar queries SQL manuais, como *SELECT*, *INSERT*, *UPDATE* e *DELETE*. Toda a persistência de dados é centralizada nessa camada, garantindo organização e isolamento da lógica SQL.

5. Camada de Middlewares

- Utilizada para:
- autenticação;
 - validação de tokens;
 - tratamento de erros;
 - registro de logs.

4.3. Implementação dos Módulos

A seguir, são detalhados os módulos do sistema e suas respectivas funcionalidades, implementadas por meio de consultas SQL manuais.

4.3.1. Módulo: Empresa

Inclui a implementação das entidades e funcionalidades:

- Empresa

Cadastro, edição, autenticação e gerenciamento geral das empresas cadastradas.

- Vaga

Cada empresa pode criar uma ou várias vagas.

A entidade vaga inclui atributos como:

- título,
- descrição,

tipo,
requisitos,
vínculo com a empresa.

- Subtipos da Vaga

Permitem classificar a vaga em categorias específicas, facilitando a busca e o match com candidatos.

- Acessibilidades da Vaga

Registro dos recursos de acessibilidade oferecidos pela vaga, como: rampas, piso tátil, intérprete de Libras, adaptações estruturais, entre outros.

Arquivos envolvidos nesse módulo:

empresa.routes.ts

empresa.controller.ts

empresa.service.ts

empresa.repository.ts (queries SQL manuais)

vaga.routes/controller/service/repository.ts

subtiposVaga.routes/controller/service/repository.ts

acessibilidadesVaga.routes/controller/service/repository.ts

4.3.2. Módulo: Candidato

- Entidade Candidato

Contém dados pessoais, perfil, qualificações e informações específicas para inclusão e acessibilidade.

- Subtipos do Candidato

Classificações que ajudam a identificar perfis apropriados para determinadas vagas.

- Barreiras do Candidato

Registro das barreiras enfrentadas, como limitações arquitetônicas, comunicacionais ou físicas.

- Match entre Candidato e Vaga

Implementado como serviço dedicado.

A lógica analisa:

compatibilidade entre subtipo da vaga e subtipo do candidato;

acessibilidades oferecidas versus barreiras declaradas;

requisitos da vaga;

perfil geral do candidato.

O resultado indica se existe compatibilidade e retorna um índice de match.

Arquivos do módulo Candidato:

candidato.routes.ts

candidato.controller.ts

candidato.service.ts

candidato.repository.ts (queries SQL manuais)

subtiposCandidato.routes/controller/service/repository.ts

barreiras.routes/controller/service/repository.ts

match.service.ts.

4.4. Testes Automatizados

Para garantir confiabilidade, foram criados testes unitários e de integração utilizando Jest, abrangendo:

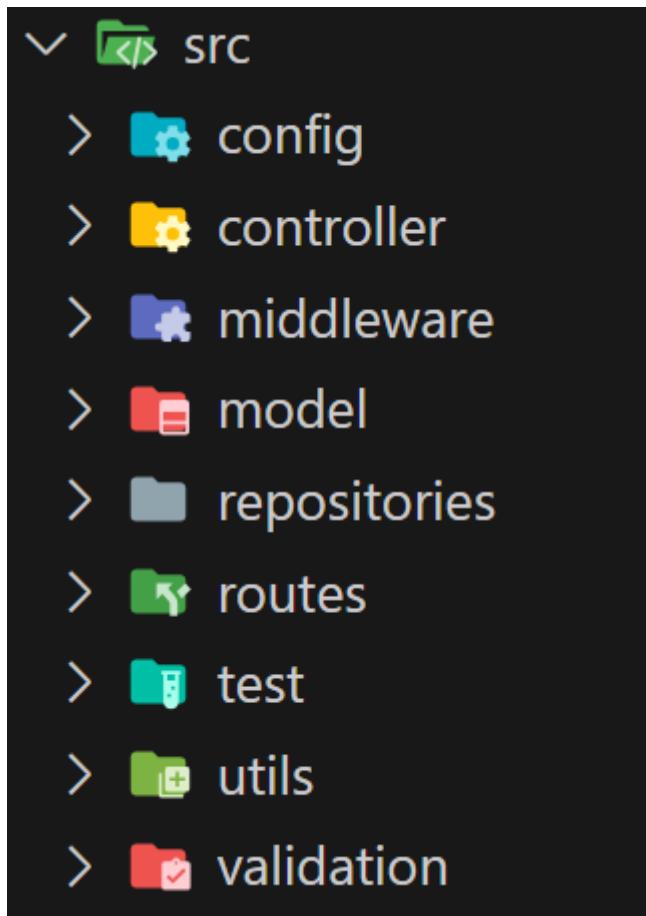
regras de negócio;

interações com o banco de dados;

integração entre camadas;

validação de fluxo das rotas.

O projeto alcançou mais de 100 testes implementados e acima de 90% de cobertura, garantindo robustez e estabilidade ao backend.



Fonte: Os autores

A arquitetura do projeto, foi pensado em promover uma melhor legibilidade nas funções e uma manutenção eficiente, separando cada estágio do desenvolvimento em camadas. Também foi utilizada o Padrão MVC, dita previamente.

Das Camadas:

- src/
 - Camada onde ficará todo e qualquer código fonte do projeto
- config/
 - Camada de configuração do “node-postgres”
- controller/

- Camada controladora, ela recebe as requisições das rotas HTTP e designa funções para as camadas seguintes
- middleware/
 - Camada de configuração das rotas, armazena códigos de validação de tokens
- model/
 - Camada que recebe funções da camada controller. A model possui as regras de negocio, refina os dados e é responsável em designar funções para camadas seguintes
- repositories/
 - Camada responsável pelas queries SQL, inserção de dados, consulta a tabelas e cláusulas SQL.
- routes/
 - Camada responsável por definir os endpoints HTTP
- test/
 - Camada de test com Jest.ts
- utils/
 - Camada responsável por definir as rotas e suas requisições, ela quem passa as requisições para a camada controller
- validation/
 - Camada responsável pela validação de dados

```
1 let passwordIsValid: boolean = newUser.senha === newUser.confirme_senha;
2 if (passwordIsValid) {
3   console.log("[POST / CRIPTOGRAFANDO SENHA]");
4   const salt = await bcrypt.genSalt(12);
5   const passwordHash = await bcrypt.hash(user.senha, salt);
6   newUser.SetCryptPass(passwordHash);
7 }
```

Também foi implementado criptografia em dados sensíveis, como as senhas de usuários. Isso permite uma camada a mais de segurança na aplicação.

```
1 export let authenticateTokenEmp = (req: any, res: any, next: any) => {
2   const authHeader = req.headers["authorization"];
3   console.log("Auth Header:", authHeader);
4   const secretEmp = process.env.SECRET_EMP as string;
5   const token = JWT.sign(
6     { id: 123, role: "empresa" },
7     secretEmp,
8     { expiresIn: "1h" }
9   );
10  console.log("Secret EMP:", secretEmp);
11  if (!token) {
12    return res.status(401).json({ msg: "Não autorizado!" });
13  }
14  try {
15    const payload = JWT.verify(token, secretEmp);
16    console.log("Payload decodificado:", payload);
17    next();
18  } catch (error) {
19    console.error("Erro JWT:", error);
20    res.status(403).json({ msg: "Token inválido!" });
21  }
22  };
```

Middlewares são utilizados para incrementar a API. Neste caso, o uso de JWT (JSON Web Tokens) tem o intuito de existir uma validação prévia antes da rota HTTP ser utilizada, logo, cada usuário pode acessar a rota que diz respeito ao seu usuário, e assim por diante.

5. IMPLEMENTAÇÃO DA SOLUÇÃO DO FRONTEND

O projeto foi construído utilizando uma stack moderna e robusta, desenvolvida com foco em performance, escalabilidade e acessibilidade. A escolha das tecnologias teve como objetivo garantir uma experiência de desenvolvimento ágil, além de entregar um produto final consistente e de alta qualidade.

A aplicação frontend foi construída com Next.js, um framework moderno baseado em React, porém muito mais completo, oferecendo recursos nativos como renderização híbrida (SSR, SSG e ISR), rotas otimizadas, melhor SEO e estruturação automática de páginas. Esses elementos contribuíram diretamente para melhorar o desempenho geral do sistema e a experiência do usuário.

O Next.js também proporcionou uma organização mais clara do projeto, facilitando a construção de interfaces reutilizáveis, a manutenção dos componentes e a integração com a API backend.

O código foi escrito em TypeScript, garantindo maior segurança durante o desenvolvimento por meio de tipagem estática, redução de erros em tempo de execução e melhoria na legibilidade e manutenção do código. Além disso, o TypeScript fornece um autocompletar inteligente, o que acelera o desenvolvimento e aumenta a confiabilidade do sistema.

Para estilização, utilizou-se o TailwindCSS, um framework CSS utility-first que permite criar interfaces personalizadas diretamente no JSX. O TailwindCSS possibilitou a construção de um design consistente, responsivo e altamente customizável, reduzindo o tempo de desenvolvimento visual e evitando repetição de estilos.

Crie sua Conta
Junte-se a milhares de profissionais e empresas inclusivas

[Sou Candidato](#) [Sou Empresa](#)

Informações Pessoais

Nome Completo *

Digite seu nome completo

Email *

seu@email.com

Confirmar Email *

Confirme seu email

CPF *

000.000.000-00

Telefone *

(00) 00000-0000

Data de Nascimento *

dd/mm/aaaa

Segurança da Conta

Senha *

Mínimo 6 caracteres

Confirmar Senha *

Confirme sua senha

A tela de cadastro foi projetada para permitir que o usuário escolha inicialmente seu tipo de perfil — *Candidato* ou *Empresa*. Após essa seleção, os campos específicos de cada tipo de conta são exibidos. O formulário foi desenvolvido com foco em acessibilidade, utilizando labels claros, espaçamento adequado, validação visual e semântica, além de elementos interativos compatíveis com navegação via teclado e leitores de tela.

Objetivo da Tela:

- Realizar o registro inicial do usuário no sistema.
- Garantir que o processo de cadastro seja simples, rápido e acessível.
- Adaptar dinamicamente os campos conforme o tipo de conta selecionado.
- Reforçar segurança com campos de confirmação de senha e email.

Tecnologias e recursos aplicados:

- Componentização com Next.js (App Router)
- Validação de formulários com TypeScript
- Estilização responsiva usando TailwindCSS
- Máscaras de entrada (CPF, telefone, data) para auxiliar usuários com baixa experiência digital

Bem-vindo de volta!

Faça login para continuar

[Candidato](#) [Empresa](#)

Email *

seu@email.com

Senha *

Digite sua senha

[Esqueceu a senha?](#)

Entrar

Não tem uma conta? [Cadastre-se](#)

A interface de login foi construída de forma simples e minimalista, priorizando clareza e agilidade no acesso. Assim como na tela de cadastro, o usuário seleciona se fará login como *Candidato* ou *Empresa*, permitindo personalizar a autenticação conforme o tipo de conta.

Objetivo da Tela:

- Permitir autenticação rápida e segura.
- Oferecer navegação simples entre os modos *Candidato* e *Empresa*.
- Fornecer uma experiência inclusiva, com campos bem definidos e compatíveis com leitores de tela.
- Garantir responsividade para acesso via dispositivos móveis.

Tecnologias e recursos aplicados:

- Formulários tipados em TypeScript
- Feedbacks visuais para erros de autenticação
- TailwindCSS para responsividade imediata

A landing page tem como objetivo apresentar o sistema, seus benefícios e sua proposta inclusiva. O layout foi projetado com alto contraste,

hierarquia visual clara e elementos grandes, garantindo acessibilidade a usuários com baixa visão ou dificuldades cognitivas.

A página contém botões de ação direta (“Começar Agora”, “Já tenho conta”), além de seções explicativas sobre o sistema, reforçando o propósito inclusivo da plataforma.

Objetivo da Tela:

- Introduzir o usuário ao sistema de maneira acolhedora e acessível.
- Facilitar a navegação para cadastro e login.
- Mostrar visualmente os valores de inclusão e acessibilidade do projeto.
- Reforçar a proposta social da plataforma PCDentro.

Tecnologias e recursos aplicados:

- Renderização otimizada com Next.js
- Componentes reutilizáveis para banners e blocos informativos
- TailwindCSS para gradientes, espaçamentos e tipografia
- Design responsivo para diferentes resoluções



Estrutura de Pastas e Organização do Frontend

A arquitetura do frontend foi implementada utilizando o App Router do Next.js, que permite uma organização clara, escalável e baseada em rotas de arquivos. Dentro do diretório principal `app/`, cada pasta representa uma rota da aplicação, e cada arquivo `page.tsx` representa a página renderizada para essa rota.

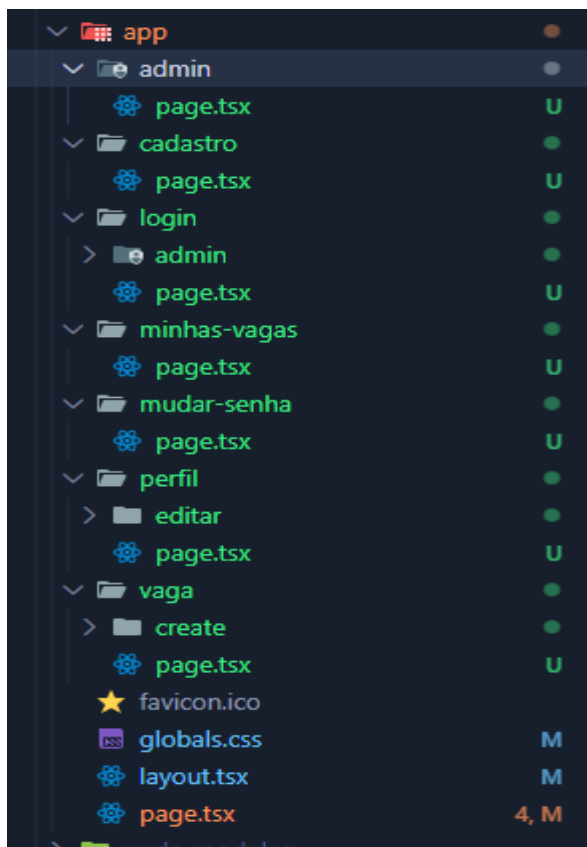
Essa estrutura facilita a manutenção, melhora a legibilidade do código e permite o uso eficiente de recursos do framework, como *Server Components*, *Client Components*, *Layouts*, *Loading States* e *Route Groups*.

A seguir, apresenta-se a descrição detalhada da organização do diretório:

Diretório app/

O diretório app/ é o núcleo do frontend, responsável pelo mapeamento das rotas e renderização das páginas. Ele contém as telas principais do sistema, organizadas por contexto de uso.

A estrutura apresentada é a seguinte:



5.2.2 Pasta admin/

Função:

Contém as páginas exclusivas do administrador da plataforma.

Arquivo principal: admin/page.tsx

Exibe a interface inicial do painel administrativo.

Permite o gerenciamento geral do sistema (usuarios, vagas etc.).

É uma rota protegida e acessível apenas para perfis administrativos.

5.2.3 Pasta cadastro/

Função:

Responsável pela página de criação de conta para candidatos e empresas.

Arquivo: cadastro/page.tsx

Exibe formulário dinâmico com campos específicos para cada tipo de usuário.

Realiza validação e integração com a API de cadastro do backend.

Utiliza componentes reutilizáveis para entradas, selects e máscaras.

5.2.4 Pasta login/

Função:

Gerencia as telas de autenticação da plataforma.

login/page.tsx

Tela de login para candidatos.

Inclui validações e feedback de erros.

Acessível e responsiva.

login/admin/page.tsx

Tela de login exclusiva para administradores.

Utiliza a mesma base visual, porém com autenticação distinta.

5.2.5 Pasta minhas-vagas/

Função:

Página onde empresas visualizam todas as vagas que já cadastraram.

Arquivo: minhas-vagas/page.tsx

Lista de vagas cadastradas.

Ações como editar, remover ou visualizar informações.

Integração com a API via requisições server-side do Next.js.

5.2.6 Pasta mudar-senha/

Função:

Tela dedicada à recuperação ou troca de senha.

Arquivo: mudar-senha/page.tsx

Formulário simples e acessível.

Inclui validação e confirmação da nova senha.

5.2.7 Pasta perfil/

Contém as telas relacionadas ao perfil do usuário.

perfil/page.tsx

Exibe informações pessoais do usuário (candidato ou empresa).

Permite visualizar dados cadastrados e realizar ações básicas.

perfil/editar/page.tsx

Tela para edição de informações do perfil.

Utiliza componentes reutilizáveis e validação detalhada.

5.2.8 Pasta vaga/

Reúne tudo relacionado ao gerenciamento de vagas.

vaga/page.tsx

Exibe detalhes de uma vaga específica.

Disponibiliza informações completas sobre acessibilidades, requisitos

etc.

vaga/create/page.tsx

Tela exclusiva para criação de vagas por empresas.

Estrutura de formulário completa e validada.

5.2.9 Arquivos Raiz do Diretório app/

page.tsx

Página inicial (Landing Page).

Apresenta a proposta do sistema e direciona o usuário ao login ou cadastro.

layout.tsx

Layout global da aplicação.

Define a estrutura padrão das páginas (header, footer, estilos globais).

Responsável por compartilhar elementos entre telas usando o App Router.

globals.css

Arquivo contendo estilos globais do TailwindCSS e reset de CSS.

favicon.ico

Ícone exibido no navegador para identificação do sistema.

5.2.10 Benefícios da Organização Utilizada

A estrutura apresentada traz vantagens como:

Escalabilidade: módulos separados por contexto facilitam expansão futura.

Coesão e clareza: cada pasta representa uma funcionalidade específica.

Manutenibilidade: componentes e páginas ficam organizados de forma clara.

Rapidez no desenvolvimento: o App Router reduz a necessidade de configuração manual de rotas.

Padronização visual: o uso de componentes e estilização utilitária garante consistência

7. CONSIDERAÇÕES FINAIS

O presente projeto de Unidade Curricular de Extensão alcançou com êxito seu objetivo geral, que consistia em capacitar os discentes no desenvolvimento de uma aplicação web completa, aplicando de forma efetiva os paradigmas de programação orientada a objetos, funcional e orientada a componentes.

Este propósito acadêmico foi materializado em uma solução de software com relevante impacto social: uma plataforma dedicada a promover a inclusão de pessoas com deficiência (PCD) no mercado de trabalho, conectando-as a oportunidades de emprego e abordando diretamente os Objetivos de Desenvolvimento Sustentável (ODS) 8, 10 e 11.

8. CONTRIBUIÇÕES DA UCE PARA A FORMAÇÃO DISCENTE

A Unidade Curricular de Extensão (UCE) representou um pilar fundamental para a nossa formação, transcendendo o aprendizado teórico de paradigmas de programação e nos imergindo em um ciclo de desenvolvimento de software completo e com propósito.

A relevância desta atividade se manifestou em múltiplas dimensões, desde a capacitação técnica até o desenvolvimento de uma consciência social mais aguçada.

Do ponto de vista técnico, a experiência foi imensamente enriquecedora. Tivemos a oportunidade de aplicar na prática os conceitos estudados em sala de aula, construindo uma solução *full-stack* com tecnologias modernas e requisitadas pelo mercado, como TypeScript, Node.js, React.js e PostgreSQL.

A implementação da arquitetura MVC, a criação de testes unitários e de integração com Jest.js, a aplicação de medidas de segurança como JWT e criptografia, e a construção de uma interface reativa e responsiva nos proporcionaram uma visão realista e prática dos desafios enfrentados no dia a dia de um desenvolvedor de software.

Essa transição da teoria para a prática solidificou nosso conhecimento e nos deu confiança em nossas habilidades técnicas.